Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 3

по дисциплине «**Основы разработки на Java**»

Выполнил:
студент гр. ИС-542
«11» октября 2026 г. _____ /Кузугашев С.В./

Проверил:
Ассистент кафедры ВС
«12» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Оглавление

Задание 3
Выполнение работы 5
Структура проекта 5
Задача 1. Один контракт, две реализации 5
Задача 2. Dynamic dispatch на бумаге 6
Задача 3. Полиморфный массив 7
Задача 4. Абстрактный базовый класс 7
Задача 5. Интерфейс или абстрактный класс 8
Задача 6. Переопределение без случайной перегрузки 9
Задача 7. Композиция вместо наследования 10
Задача 8. Нарушение LSP 10
Задача 9. Политика допуска как стратегия 11
Задача 10. Конфликт default-методов 12
Задача 11. Равенство в иерархии 12
Задача 12. Sealed-иерархия результата 13
Задача 13. Удаление type code 14
Задача 14. Наследование взрывает комбинации 14
Задача 15. Расширяемый движок правил 15
Ошибки и их исправление 17
Заключение 18
Приложение 19

Задание

Дисциплина «Программирование». Лабораторная работа №3. Тема: полиморфизм и проектирование на Java — динамическая диспетчеризация, интерфейсы и абстрактные классы, композиция, sealed-иерархии и расширяемый движок правил. Среда: OpenJDK 21.

Исходный код лабораторной работы опубликован в открытом репозитории: https://github.com/Forlein/SibGUTI-Java-Labs (папка WORK_Java/laboratory_work_3).

В работе реализованы 15 заданий:

1. Один контракт, две реализации — интерфейс CourseFormatter, реализации CompactCourseFormatter и DetailedCourseFormatter, вызов через интерфейсный тип.

2. Dynamic dispatch на бумаге — иерархия NotificationSender → EmailSender, ConsoleSender и разбор выбора перегрузки и переопределения.

3. Полиморфный массив — массив интерфейсного типа Socket и обход циклом for-each без if по типу.

4. Абстрактный базовый класс — final-метод send с общей проверкой адресата и абстрактная точка расширения doSend.

5. Интерфейс или абстрактный класс — выбор контракта для Identifiable, Formatter и BaseEntity.

6. Переопределение без случайной перегрузки — роль @Override при смене типа параметра.

7. Композиция вместо наследования — декораторы LoggingSender и RetrySender и два порядка оборачивания.

8. Нарушение LSP — разделение интерфейсов CourseReader и CourseWriter.

9. Политика допуска как стратегия — интерфейс EnrollmentPolicy и его реализации.

10. Конфликт default-методов — ошибка компиляции и разрешение через Interface.super.send().

11. Равенство в иерархии — симметрия equals для Point и ColoredPoint и два исправления.

12. Sealed-иерархия результата — исчерпывающий switch без ветки default.

13. Удаление type code — замена switch по строке на полиморфный интерфейс.

14. Наследование взрывает комбинации — две независимые оси Format и Payment через композицию.

15. Расширяемый движок правил — sealed RuleResult, интерфейс Rule и комбинаторы AllOf/AnyOf.

Работа выполнена на оценку «5»: реализованы все 15 заданий, каждое собрано и запущено, программы выдают корректный результат.

Выполнение работы

Ниже по каждой задаче приведён ключевой фрагмент кода (микро-листинг) и вывод программы; полные исходники лежат в папках 1–15 проекта.

Структура проекта

Каждое задание вынесено в отдельную папку с номером, внутри которой лежат исходники .java. Классы с одинаковыми именами (Course, Main, NotificationSender и т. п.) продублированы в папках, поэтому задания собираются и запускаются по отдельности. Сборка: javac -encoding UTF-8 -d out <папка>*.java; запуск: java -Dstdout.encoding=UTF-8 -cp out <Класс>.

Папка	Файлы и задание
1	Proba_1.java — один контракт, две реализации
2	Main.java — dynamic dispatch на бумаге
3	Main.java — полиморфный массив
4	Main.java — абстрактный базовый класс
5	Main.java — интерфейс или абстрактный класс
6	Main.java — переопределение без случайной перегрузки
7	Main.java — композиция вместо наследования
8	Main.java — нарушение LSP
9	Main.java — политика допуска как стратегия
10	main_1.java, main_2.java — конфликт default-методов
11	Main_1.java, Main_2.java — равенство в иерархии
12	Main.java — sealed-иерархия результата
13	Main.java — удаление type code
14	Main.java — наследование взрывает комбинации
15	Main.java — расширяемый движок правил
Корень	README.md, Отчет_ЛР3_Java.docx

Задача 1. Один контракт, две реализации

В первой задаче объявлен общий интерфейс CourseFormatter с методом format(Course) и созданы две независимые реализации: CompactCourseFormatter и DetailedCourseFormatter. Обе реализации вызываются через переменную интерфейсного типа, поэтому клиентский код не знает о конкретном классе.

Листинг – 1 Вызов реализаций через интерфейсную ссылку (1/Proba_1.java)

```
1     Course c = new Course();
2     CourseFormatter f = new CompactCourseFormatter();
3     System.out.print(f.format(c));
4     f = new DetailedCourseFormatter();
5     System.out.print(f.format(c));
```

Вывод программы (1/Proba_1)

```
1     Консоль: 12  (Compact вывел "1", Detailed вывел "2" без перевода
      строки)
```

Компилятору известен только статический тип переменной CourseFormatter, поэтому он проверяет лишь наличие метода format(Course). Во время выполнения JVM смотрит на фактический объект и вызывает именно его реализацию — это и есть динамическая диспетчеризация.

Задача 2. Dynamic dispatch на бумаге

Во второй задаче построена иерархия NotificationSender → EmailSender, ConsoleSender с перегруженными методами send(Object) и send(String). Наследники переопределяют только send(String), а вызовы выполняются через ссылки разных статических типов.

Листинг – 2 Перегрузка и переопределение (2/Main.java)

```
1     NotificationSender s1 = new EmailSender();
2     NotificationSender s2 = new ConsoleSender();
3     EmailSender e = new EmailSender();
4     s1.send("hello");          // override -> Email.send(String)
5     s2.send("hello");          // override -> Console.send(String)
6     s1.send(42);               // autoboxing -> send(Object), не
      переопределён
7     s1.send((Object) "hello");  // явное приведение -> send(Object)
```

Вывод программы (2/Main)

```
1     --- 1. Обычный String ---
2     Email.send(String): hello
3     Console.send(String): hello
4     Email.send(String): hello
5     --- 2. Число int (упаковывается в Object) ---
6     Base.send(Object): 42
7     Base.send(Object): 42
```

```
8     --- 3. Явное приведение к Object ---
9     Base.send(Object): hello
10    Base.send(Object): hello
```

Overload (перегрузка) выбирается компилятором по статическому типу ссылки и аргументов, а override (переопределение) — JVM по фактическому типу объекта. Поэтому send(42) и send((Object) "hello") уходят в не переопределённый метод родителя Base.send(Object), а send("hello") вызывает реализацию наследника.

Задача 3. Полиморфный массив

В третьей задаче несколько реализаций интерфейса Socket сложены в массив интерфейсного типа и обработаны единым циклом for-each без операторов if, switch или instanceof по типу устройства.

Листинг – 3 Полиморфный обход массива (3/Main.java)

```
1     Socket[] devices = {
2         new Iron(),
3         new Phone(),
4         new Kettle()
5     };
6     for (Socket device : devices) {
7         System.out.println(device.doWork(c));
8     }
```

Вывод программы (3/Main)

```
1     1
2     2
3     3
```

Клиентский цикл зависит только от интерфейса Socket и не содержит проверок типа. Чтобы добавить новую реализацию, достаточно создать класс и добавить его экземпляр в массив — сам цикл менять не нужно, что демонстрирует принцип «открыт для расширения, закрыт для изменения».

Задача 4. Абстрактный базовый класс

В четвёртой задаче создан абстрактный класс AbstractNotificationSender, реализующий шаблонный метод. Публичный метод send(address, message) объявлен final: он выполняет общую проверку адресата, затем вызывает

абстрактный doSend и логирует результат. Подклассы переопределяют только doSend.

Листинг – 4 Шаблонный метод и точка расширения (4/Main.java)

```
1     public final void send(String address, String message) {
2         if (address == null || address.isBlank()) {
3             System.out.println("Ошибка: адресат не указан!");
4             return;
5         }
6         doSend(address, message);
7         System.out.println("Лог: сообщение для " + address + "
      обработано.");
8     }
9     protected abstract void doSend(String address, String message);
```

Вывод программы (4/Main)

```
1     --- Тест 1: нормальный адрес ---
2     Email отправлен на user@mail.com: Привет
3     Лог: сообщение для user@mail.com обработано.
4     Вывод в консоль [admin]: Система запущена
5     Лог: сообщение для admin обработано.
6     --- Тест 2: пустой адрес (проверка родителя) ---
7     Ошибка: адресат не указан!
```

Модификатор final у публичного метода send() не даёт подклассам переопределить общий алгоритм и обойти проверку адресата. Точкой расширения остаётся protected abstract doSend(), поэтому наследники добавляют только специфичную доставку, не дублируя проверку.

Задача 5. Интерфейс или абстрактный класс

В пятой задаче для архитектурных понятий выбран вид контракта: Identifiable и Formatter объявлены интерфейсами (чистое поведение), а BaseEntity — абстрактным классом, который хранит общее поле id и готовую реализацию getId().

Листинг – 5 Интерфейс, абстрактный класс и наследник (5/Main.java)

```
1     interface Identifiable { long getId(); }
2     abstract class BaseEntity implements Identifiable {
3         protected long id;
4         public long getId() { return id; }
5     }
```

```
6     class Student extends BaseEntity { private String name; ... }
7     interface Formatter { String format(Student student); }
```

Вывод программы (5/Main)

```
1     --- ДЕМОНСТРАЦИЯ КОДА ---
2     ID студента (из BaseEntity): 101
3     Форматирование (из Formatter): Студент: Иван
4     • Нужно только поведение (методы)? -> Делай INTERFACE.
5     • Нужны общие поля и готовый код? -> Делай ABSTRACT CLASS.
6     • Нужна конкретная сущность с данными? -> Делай обычный CLASS.
```

Выбор абстрактного класса для BaseEntity избавляет все сущности от дублирования поля id и кода getId(), но расходует единственное наследование extends. Интерфейсы Identifiable и Formatter задают только поведение, поэтому их можно комбинировать в любом количестве.

Задача 6. Переопределение без случайной перегрузки

В шестой задаче показана ловушка: в классе Iron объявлен метод doWork(Course), а в наследнике IronBroken без аннотации @Override написан метод doWork(Object). Изменённый тип параметра делает метод перегрузкой, а не переопределением.

Листинг – 6 Случайная перегрузка вместо переопределения (6/Main.java)

```
1     class IronBroken extends Iron {
2         // Нет @Override, параметр Object вместо Course — это overload!
3         public String doWork(Object course) {
4             return "ОШИБКА: сработал не тот метод!";
5         }
6     }
7     broken.doWork(c);           // -> Iron.doWork(Course): правильный
      метод
8     broken.doWork((Object) c);  // -> IronBroken.doWork(Object): ловушка
```

Вывод программы (6/Main)

```
1     1 (Правильный метод)
2     ОШИБКА: сработал не тот метод!
```

Без @Override компилятор не проверяет, что метод действительно переопределяет родительский. Изменение типа параметра превращает переопределение в перегрузку, и через ссылку родительского типа вызывается

не тот метод. Аннотация @Override заставила бы компилятор сообщить об ошибке ещё до запуска.

Задача 7. Композиция вместо наследования

В седьмой задаче логирование и повторные попытки реализованы декораторами LoggingSender и RetrySender, которые реализуют NotificationSender и принимают другой NotificationSender в конструктор. Показаны два разных порядка оборачивания.

Листинг – 7 Декораторы и два порядка композиции (7/Main.java)

```
1     NotificationSender order1 = new LoggingSender(new
      RetrySender(baseEmail));
2     NotificationSender order2 = new RetrySender(new
      LoggingSender(baseEmail));
3     order1.send("Привет"); // логирование снаружи, повторы внутри
4     order2.send("Привет"); // повторы снаружи, логирование внутри
```

Вывод программы (7/Main)

```
1     --- Порядок 1: Сначала Логирование, потом Retry ---
2     [LOG] Начинаю отправку: Привет
3       [EmailSender] Письмо отправлено: Привет
4     [LOG] Отправка завершена.
5     --- Порядок 2: Сначала Retry, потом Логирование ---
6     [LOG] Начинаю отправку: Привет
7       [EmailSender] Письмо отправлено: Привет
8     [LOG] Отправка завершена.
```

Декораторы оборачивают друг друга и делегируют работу внутреннему объекту, а не наследуют его. Порядок вложенности задаёт поведение: логирование снаружи срабатывает один раз на весь вызов, а логирование внутри повторов выполнялось бы на каждой попытке. Композиция позволяет собирать поведение в любом порядке без классов-мутантов.

Задача 8. Нарушение LSP

В восьмой задаче контракт репозитория разделён на CourseReader (только чтение) и CourseWriter (чтение и запись). Реализация ReadOnlyRepository обещает только чтение и не выбрасывает исключение из несуществующего метода save().

Листинг – 8 Разделение интерфейсов чтения и записи (8/Main.java)

```
interface CourseReader { String findById(int id); }
interface CourseWriter extends CourseReader { void save(String course); }
class DatabaseRepository implements CourseWriter { ... }
class ReadOnlyRepository implements CourseReader { ... }
CourseReader reader = new ReadOnlyRepository();
writer.save("Java");
```

Вывод программы (8/Main)

```
Курс из кэша
Сохранено в БД: Java
Курс из БД
```

Подтип не должен ломать контракт базового типа. Если бы ReadOnlyRepository реализовывал общий интерфейс с методом save(), ему пришлось бы бросать UnsupportedOperationException, что нарушает принцип подстановки Лисков. Разделение интерфейсов позволяет read-only источнику честно реализовывать только чтение.

Задача 9. Политика допуска как стратегия

В девятой задаче объявлен интерфейс стратегии EnrollmentPolicy и три реализации: AgePolicy, PrereqPolicy и CapacityPolicy. Сервис EnrollmentService принимает политику через конструктор и не знает о конкретных правилах.

Листинг – 9 Внедрение стратегии через конструктор (9/Main.java)

```
interface EnrollmentPolicy { boolean isAllowed(Student s, Course c); }
class AgePolicy implements EnrollmentPolicy {
    public boolean isAllowed(Student s, Course c) { return s.age >= 18; }
}
EnrollmentService service = new EnrollmentService(new AgePolicy());
service.enroll(kid, c);   // Отказ
service.enroll(adult, c); // Записан
```

Вывод программы (9/Main)

```
Отказ!
Записан!
```

Сервис зачисления знает только интерфейс EnrollmentPolicy и делегирует ему проверку, поэтому подключение новой политики не требует

изменения EnrollmentService. Это паттерн «Стратегия»: алгоритм допуска вынесен в отдельный объект и подставляется через конструктор, что делает код гибким и тестируемым.

Задача 10. Конфликт default-методов

В десятой задаче два интерфейса EmailNotifier и SmsNotifier содержат одинаковый default-метод send(). Попытка реализовать оба интерфейса без переопределения приводит к ошибке компиляции, а конфликт разрешается явным вызовом Interface.super.

Листинг – 10 Конфликт и его разрешение (10/main_1.java, 10/main_2.java)

```
1    interface EmailNotifier { default void send() { ... } }
2    interface SmsNotifier  { default void send() { ... } }
3    // class MultiNotifier implements EmailNotifier, SmsNotifier {} —
     ошибка компиляции
4    class MultiNotifier implements EmailNotifier, SmsNotifier {
5        @Override public void send() {
6            EmailNotifier.super.send();
7            SmsNotifier.super.send();
8        }
9    }
```

Вывод программы (10/main_1.java, 10/main_2.java)

```
1    javac: class MultiNotifier inherits unrelated defaults for send()
     from EmailNotifier and SmsNotifier
2    Отправка по Email
3    Отправка по SMS
```

Ромбовидная неоднозначность при множественном наследовании default-методов разрешается не автоматически: Java требует явно переопределить метод и указать, чьи реализации вызывать. Это защищает от скрытого выбора произвольной ветки.

Задача 11. Равенство в иерархии

В одиннадцатой задаче воспроизведено нарушение симметричности equals между Point и ColoredPoint, а затем показаны два решения: строгое сравнение по классу (getClass) и замена наследования композицией цвета.

Листинг – 11 Нарушение симметрии и исправления (11/Main_1.java, 11/Main_2.java)

```
1     // Main_1: p.equals(cp)=true, cp.equals(p)=false -> симметрия
      нарушена
2     if (!(o instanceof Point)) return false;        // базовый класс
3     if (!(o instanceof ColoredPoint)) return false; // наследник
4     // Main_2: строгое сравнение по классу
5     if (o == null || getClass() != o.getClass()) return false;
6     class ColoredPoint { private final Point point; ... } // композиция
```

Вывод программы (11/Main_1.java, 11/Main_2.java)

```
1     p.equals(cp): true
2     cp.equals(p): false
3     Симметрия нарушена!
4     p.equals(cp): false
5     cp.equals(p): false
6     Симметрия соблюдена!
```

Если подкласс добавляет значимое поле в equals, симметрия ломается: p.equals(cp) возвращает true, а cp.equals(p) — false. Строгое сравнение по классу сохраняет симметрию, но нарушает принцип подстановки; композиция, когда ColoredPoint содержит Point внутри, решает проблему без нарушения контрактов.

Задача 12. Sealed-иерархия результата

В двенадцатой задаче объявлен sealed interface EnrollmentResult с тремя разрешёнными record-реализациями Accepted, Rejected и WaitListed. Все варианты обрабатываются исчерпывающим switch без ветки default.

Листинг – 12 Sealed-интерфейс и exhaustive switch (12/Main.java)

```
1     sealed interface EnrollmentResult permits Accepted, Rejected,
      WaitListed {}
2     record Accepted(String courseName) implements EnrollmentResult {}
3     record Rejected(String reason) implements EnrollmentResult {}
4     record WaitListed(int position) implements EnrollmentResult {}
5     switch (result) {
6         case Accepted a -> ...;
7         case Rejected r -> ...;
8         case WaitListed w -> ...;
9     } // default не нужен: компилятор знает все варианты
```

Вывод программы (12/Main)

```
1     Успех! Вы записаны на Java
2     Отказ. Причина: Нет мест
3     В листе ожидания. Ваш номер: 5
```

sealed ограничивает круг наследников и даёт компилятору полный перечень вариантов. Компилятор требует обработать их все, и при добавлении нового типа switch перестанет компилироваться — так ошибка о непокрытом варианте обнаруживается ещё на этапе компиляции.

Задача 13. Удаление type code

В тринадцатой задаче switch по строковому типу заменён на полиморфный интерфейс NotificationSender с реализациями EmailSender, SmsSender и PushSender. Клиентский сервис только вызывает send() и не содержит ветвлений по типу.

Листинг – 13 Полиморфные каналы доставки (13/Main.java)

```
1     interface NotificationSender { void send(String message); }
2     class EmailSender implements NotificationSender { ... }
3     class SmsSender implements NotificationSender { ... }
4     class PushSender implements NotificationSender { ... }
5     class NotificationService {
6         public void send(NotificationSender sender, String message) {
      sender.send(message); }
7     }
```

Вывод программы (13/Main)

```
1     Email: Привет
2     SMS: Привет
3     Push: Привет
```

Полиморфизм заменяет switch по строке: каждое поведение инкапсулировано в классе, а клиент зависит только от интерфейса. Добавление нового канала не требует изменения сервиса, поэтому полиморфизм оправдан там, где меняется поведение, а не просто данные конфигурации.

Задача 14. Наследование взрывает комбинации

В четырнадцатой задаче две независимые оси курса — формат (Format: Online, Classroom, Hybrid) и оплата (Payment: Free, Fixed, Subscription) —

выделены в отдельные интерфейсы и собираются композицией в классе Course.

Листинг – 14 Композиция двух независимых осей (14/Main.java)

```
interface Format  { String name(); }   // Online, Classroom, Hybrid
interface Payment { String name(); }   // Free, Fixed, Subscription
class Course {
    String title; Format format; Payment payment;
    void printInfo() { System.out.println(title + " | " +
format.name() + " | " + payment.name()); }
}
new Course("Java", new Online(), new Free()).printInfo();
```

Вывод программы (14/Main)

```
Java | Online | Free
Python | Classroom | Subscription
C# | VR | Free
Rust | Online | Crypto
```

При наследовании для M форматов и N политик оплаты потребовалось бы M × N подклассов (для 3 × 3 = 9), а добавление новой оси умножалось бы дальше. Композиция позволяет собирать любые комбинации из M + N классов: новый формат VR или оплата Crypto добавляются одним классом без правки Course.

Задача 15. Расширяемый движок правил

В пятнадцатой задаче спроектирован движок правил зачисления: закрытый sealed RuleResult (Ok, Fail), открытый интерфейс правила Rule и комбинаторы AllOf и AnyOf, хранящие массив правил. Сервис обрабатывает результат через switch.

Листинг – 15 Комбинаторы allOf/anyOf и разбор результата (15/Main.java)

```
sealed interface RuleResult permits Ok, Fail {}
record Ok() implements RuleResult {}
record Fail(String reason) implements RuleResult {}
class AllOf implements Rule { ... } // все правила должны пройти
class AnyOf implements Rule { ... } // достаточно одного
RuleResult result = rule.check(student);
switch (result) {
```

```
8           case Ok o -> ...;
9           case Fail f -> ...;
10      }
```

Вывод программы (15/Main)

```
1       --- AllOf ---
2       ОТКАЗ: Вася (Слишком молод)
3       ОТКАЗ: Иван (Нет денег)
4       УСПЕХ: Петя записан!
5       --- AnyOf ---
6       УСПЕХ: Вася записан!
7       УСПЕХ: Иван записан!
```

Наследование уместно для RuleResult — неизменяемой структуры данных с жёстким набором вариантов, a sealed гарантирует, что switch покроет их все. Композиция используется в AllOf и AnyOf: они хранят массив правил и не знают конкретных типов. Сервис зависит только от интерфейса Rule и не проверяет классы через instanceof.

Ошибки и их исправление

Классы с одинаковыми именами при общей сборке (структура). Классы Course, Main, NotificationSender повторяются в разных папках, и при совместной компиляции возникает ошибка duplicate class. Каждое задание вынесено в отдельную папку и собирается независимо.

Конфликт default-методов (задача 10). Файл main_1.java не компилируется: 'class MultiNotifier inherits unrelated defaults for send() from EmailNotifier and SmsNotifier'. Конфликт разрешён в main_2.java явным переопределением send() с вызовом EmailNotifier.super.send() и SmsNotifier.super.send().

Случайная перегрузка вместо переопределения (задача 6). Параметр Object вместо Course без аннотации @Override превратил переопределение в перегрузку, и через ссылку родительского типа вызывался родительский метод. Исправление — сигнатура doWork(Course) и аннотация @Override.

Нарушение симметрии equals в иерархии Point/ColoredPoint (задача 11). Из-за instanceof в базовом и дочернем классах p.equals(cp) давало true, а cp.equals(p) — false. Исправлено строгим сравнением getClass() != o.getClass() и заменой наследования композицией цвета.

Исчерпывающий switch в sealed-иерархии (задача 12). При добавлении нового record в permits компилятор указывает на неполный switch, поэтому ветка default не требуется и не может скрыть непокрытый вариант.

Кодировка русского текста при компиляции и выводе в консоль Windows. Для корректного отображения кириллицы сборка выполняется с флагом -encoding UTF-8, а запуск JVM — с параметром -Dstdout.encoding=UTF-8.

Заключение

В ходе выполнения лабораторной работы №3 были изучены и применены на практике механизмы полиморфизма, абстракции, наследования, композиции и проектирования контрактов в языке Java (OpenJDK 21).

В задачах 1–5 разобраны интерфейсы и абстрактные классы: показано различие между статическим типом ссылки (compile-time) и фактическим классом объекта (runtime), отделено разрешение перегрузки от динамической диспетчеризации, продемонстрирован полиморфный массив без if и instanceof, а также шаблонный метод с final-алгоритмом и абстрактной точкой расширения.

В задачах 6–10 исследованы правила переопределения методов и контракты подтипов: с помощью @Override предотвращена случайная перегрузка, наследование заменено цепочкой декораторов, принцип подстановки Лисков восстановлен разделением интерфейсов чтения и записи, реализован паттерн «Стратегия», а конфликт default-методов разрешён явным вызовом Interface.super.

В задачах 11–15 решены задачи архитектурного проектирования: восстановлена симметричность equals через строгое сравнение классов и композицию, создана закрытая sealed-иерархия с исчерпывающим switch, удалён type code, устранён комбинаторный взрыв разделением независимых осей и построен расширяемый движок правил на комбинаторах AllOf и AnyOf.

Все 15 заданий размещены в отдельных папках, каждое задание компилируется и выполняется без ошибок. Работа выполнена в полном объёме.

Приложение

Листинг – 16 1/Proba_1.java

```
class Course{}

    interface CourseFormatter {
        String format(Course course);
    }

    class CompactCourseFormatter implements CourseFormatter {
        @Override
        public String format(Course course) {
            return "1";
        }
    }

    class DetailedCourseFormatter implements CourseFormatter {
        @Override
        public String format(Course course) {
            return "2";
        }
    }

public class Proba_1 {

    public static void main(String[] args){

        Course c = new Course();

        CourseFormatter f = new CompactCourseFormatter();
        System.out.print(f.format(c));

        f = new DetailedCourseFormatter();
        System.out.print(f.format(c));

    }

}
```

Листинг – 17 2/Main.java

```
// 1. Базовый класс
class NotificationSender {
    public void send(Object msg) {
```

```
4             System.out.println("Base.send(Object): " + msg);
5         }
6
7         public void send(String msg) {
8             System.out.println("Base.send(String): " + msg);
9         }
10    }
11
12    // 2. Наследник Email
13    class EmailSender extends NotificationSender {
14        @Override
15        public void send(String msg) {
16            System.out.println("Email.send(String): " + msg);
17        }
18    }
19
20    // 3. Наследник Console
21    class ConsoleSender extends NotificationSender {
22        @Override
23        public void send(String msg) {
24            System.out.println("Console.send(String): " + msg);
25        }
26    }
27
28    // 4. Запуск
29    public class Main {
30        public static void main(String[] args) {
31            NotificationSender s1 = new EmailSender();
32            NotificationSender s2 = new ConsoleSender();
33            EmailSender e = new EmailSender();
34
35            System.out.println("--- 1. Обычный String ---");
36            s1.send("hello");
37            s2.send("hello");
38            e.send("hello");
39
40            System.out.println("--- 2. Число int (упаковывается в
      Object) ---");
41            s1.send(42);
42            e.send(42);
43
44            System.out.println("--- 3. Явное приведение к Object ---");
45            s1.send((Object) "hello");
46            e.send((Object) "hello");
47        }
```

```
48    }
```

Листинг – 18 3/Main.java

```
1     class Course { } // Просто "что-то", что мы форматируем
2
3     // Это РОЗЕТКА. Правило: "Кто хочет работать от сети, должен уметь
      делать format"
4     interface Socket {
5         String doWork(Course course);
6     }
7
8     // Прибор 1: Утюг (Печатает 1)
9     class Iron implements Socket {
10        @Override
11        public String doWork(Course course) {
12            return "1";
13        }
14    }
15
16    // Прибор 2: Телефон (Печатает 2)
17    class Phone implements Socket {
18        @Override
19        public String doWork(Course course) {
20            return "2";
21        }
22    }
23
24    // Прибор 3: Чайник (Печатает 3)
25    class Kettle implements Socket {
26        @Override
27        public String doWork(Course course) {
28            return "3";
29        }
30    }
31
32    public class Main {
33        public static void main(String[] args) {
34            Course c = new Course();
35
36            // Массив "розеток", в которые воткнуты разные приборы
37            Socket[] devices = {
38                new Iron(),
39                new Phone(),
40                new Kettle()
41            };
```

```
42
43          // Просто включаем каждый по очереди. Без if-ов!
44          for (Socket device : devices) {
45              System.out.println(device.doWork(c));
46          }
47      }
48  }
```

Листинг – 19 4/Main.java

```
1   // 1. Абстрактный базовый класс (каркас)
2   abstract class AbstractNotificationSender {
3
4       // ПУБЛИЧНЫЙ АЛГОРИТМ. Он final, чтобы наследники его не
    сломали.
5       public final void send(String address, String message) {
6           // Шаг 1: Общая проверка (её пишем ОДИН раз здесь)
7           if (address == null || address.isBlank()) {
8               System.out.println("Ошибка: адресат не указан!");
9               return;
10          }
11
12          // Шаг 2: Специфичная отправка (её делают наследники)
13          doSend(address, message);
14
15          // Шаг 3: Еще одна общая часть (например, логирование)
16          System.out.println("Лог: сообщение для " + address + "
    обработано.");
17      }
18
19      // АБСТРАКТНЫЙ МЕТОД. Точка расширения. У него нет тела, только
    контракт.
20      protected abstract void doSend(String address, String message);
21  }
22
23  // 2. Реализация для Email
24  class EmailSender extends AbstractNotificationSender {
25      @Override
26      protected void doSend(String address, String message) {
27          // Тут только специфика Email. Проверку адресата писать НЕ
    НУЖНО.
28          System.out.println("Email отправлен на " + address + ": " +
    message);
29      }
30  }
31
32  // 3. Реализация для Console
```

```
33  class ConsoleSender extends AbstractNotificationSender {
34      @Override
35      protected void doSend(String address, String message) {
36          // Тут только специфика Console.
37          System.out.println("Вывод в консоль [" + address + "]: " +
    message);
38      }
39  }
40
41  // 4. Проверка
42  public class Main {
43      public static void main(String[] args) {
44          AbstractNotificationSender email = new EmailSender();
45          AbstractNotificationSender console = new ConsoleSender();
46
47          System.out.println("--- Тест 1: нормальный адрес ---");
48          email.send("user@mail.com", "Привет");
49          console.send("admin", "Система запущена");
50
51          System.out.println("\n--- Тест 2: пустой адрес (проверка
    родителя) ---");
52          email.send("", "Это сообщение не должно уйти");
53      }
54  }
```

Листинг – 20 5/Main.java

```
1   // 1. Интерфейс — это контракт (Только методы, без полей)
2   interface Identifiable {
3       long getId();
4   }
5
6   // 2. Абстрактный класс — это каркас (Есть поля и общий код)
7   abstract class BaseEntity implements Identifiable {
8       protected long id; // Общее поле для всех сущностей
9
10      public BaseEntity(long id) {
11          this.id = id;
12      }
13
14      // Уже готовая реализация метода из интерфейса
15      @Override
16      public long getId() {
17          return id;
18      }
19  }
```

```
20
21  // 3. Обычный класс — готовая сущность (Наследует каркас)
22  class Student extends BaseEntity {
23      private String name;
24
25      public Student(long id, String name) {
26          super(id); // Передаем id в BaseEntity
27          this.name = name;
28      }
29
30      public String getName() { return name; }
31  }
32
33  // 4. Еще один интерфейс — чистое поведение (Форматтер)
34  interface Formatter {
35      String format(Student student);
36  }
37
38  class ShortFormatter implements Formatter {
39      @Override
40      public String format(Student s) {
41          return "Студент: " + s.getName();
42      }
43  }
44
45  public class Main {
46      public static void main(String[] args) {
47          System.out.println("--- ДЕМОНСТРАЦИЯ КОДА ---\n");
48
49          // Создаем обычный класс (Student), который наследует
    абстрактный (BaseEntity)
50          Student student = new Student(101, "Иван");
51
52          // Используем интерфейс (Formatter) для форматирования
53          Formatter formatter = new ShortFormatter();
54
55          System.out.println("ID студента (из BaseEntity): " +
    student.getId());
56          System.out.println("Форматирование (из Formatter): " +
    formatter.format(student));
57
58
    System.out.println("\n=============================================
    =========");
59          System.out.println("              ОБЪЯСНЕНИЕ (ЧИТАЙ ТУТ)");
```

```
60
    System.out.println("=============================================
    =======\n");
61
62          System.out.println("1. INTERFACE (Интерфейс) — это ПРАВИЛО
    (контракт).");
63          System.out.println("   Пример: Identifiable, Formatter.");
64          System.out.println("   Суть: Говорит 'ЧТО' нужно уметь
    делать (getId, format), но не говорит 'КАК'.");
65          System.out.println("   Зачем: Чтобы класс мог иметь много
    таких правил сразу (в Java нельзя наследовать два класса, а
    интерфейсов — сколько угодно).\n");
66
67          System.out.println("2. ABSTRACT CLASS (Абстрактный класс) —
    это КАРКАС.");
68          System.out.println("   Пример: BaseEntity.");
69          System.out.println("   Суть: Умеет хранить данные (поля,
    например 'id') и содержит готовый код, но может иметь 'дырки'
    (абстрактные методы).");
70          System.out.println("   Зачем: Чтобы не дублировать код.
    Написал один раз в BaseEntity — и все наследники (Student) этим
    пользуются.\n");
71
72          System.out.println("3. ОБЫЧНЫЙ CLASS (Обычный класс) — это
    ГОТОВАЯ ВЕЩЬ.");
73          System.out.println("   Пример: Student.");
74          System.out.println("   Суть: От него можно создать объект
    через 'new'. Он наследует каркас (BaseEntity) и выполняет правила
    (Identifiable).\n");
75
76          System.out.println("--- ГЛАВНОЕ ПРАВИЛО ВЫБОРА ---");
77          System.out.println("• Нужно только поведение (методы)? ->
    Делай INTERFACE.");
78          System.out.println("• Нужны общие поля и готовый код? ->
    Делай ABSTRACT CLASS.");
79          System.out.println("• Нужна конкретная сущность с данными? -
    > Делай обычный CLASS.\n");
80      }
81  }
```

Листинг – 21 6/Main.java

```
1   class Course { }
2
3   interface Socket {
4       String doWork(Course course);
5   }
6
7   // Родительский класс (нормальный прибор)
8   class Iron implements Socket {
```

```
9         @Override
10        public String doWork(Course course) {
11            return "1 (Правильный метод)";
12        }
13    }
14
15    // Класс-ловушка (наследник)
16    class IronBroken extends Iron {
17
18        // ТУТ ОШИБКА: Нет @Override. И параметр Object вместо Course.
19        // Компилятор думает, что это просто новый метод.
20        public String doWork(Object course) {
21            return "ОШИБКА: сработал не тот метод!";
22        }
23    }
24
25    public class Main {
26        public static void main(String[] args) {
27            Course c = new Course();
28            IronBroken broken = new IronBroken();
29
30            // Вызов 1: Передаем Course. Компилятор выбирает
      doWork(Course).
31            // Сработает метод из родителя (Iron), потому что в
      наследнике его нет.
32            System.out.println(broken.doWork(c));
33            // ВЫВОД: 1 (Правильный метод)
34
35            // Вызов 2: Передаем Object. Компилятор выбирает
      doWork(Object).
36            // Сработает метод-ловушка из наследника.
37            System.out.println(broken.doWork((Object) c));
38            // ВЫВОД: ОШИБКА: сработал не тот метод!
39        }
40    }
```

Листинг – 22 7/Main.java

```
1     // 1. Контракт
2     interface NotificationSender {
3         void send(String message);
4     }
5
6     // 2. Базовая реализация (тот самый EmailSender)
7     class EmailSender implements NotificationSender {
8         @Override
9         public void send(String message) {
```

```
10          System.out.println("  [EmailSender] Письмо отправлено: " +
    message);
11      }
12  }
13
14  // 3. Декоратор Логирования (Logging)
15  class LoggingSender implements NotificationSender {
16      private final NotificationSender wrapped; // Тот, кого мы
    оборачиваем
17
18      public LoggingSender(NotificationSender wrapped) {
19          this.wrapped = wrapped;
20      }
21
22      @Override
23      public void send(String message) {
24          System.out.println("[LOG] Начинаю отправку: " + message);
25          wrapped.send(message); // Делегируем работу внутреннему
    объекту
26          System.out.println("[LOG] Отправка завершена.");
27      }
28  }
29
30  // 4. Декоратор Повторов (Retry)
31  class RetrySender implements NotificationSender {
32      private final NotificationSender wrapped;
33
34      public RetrySender(NotificationSender wrapped) {
35          this.wrapped = wrapped;
36      }
37
38      @Override
39      public void send(String message) {
40          int attempts = 3;
41          for (int i = 1; i <= attempts; i++) {
42              try {
43                  wrapped.send(message);
44                  return; // Если успешно, выходим
45              } catch (Exception e) {
46                  System.out.println("[RETRY] Попытка " + i + " не
    удалась. Пробую снова...");
47              }
48          }
49      }
50  }
51
```

```
52  public class Main {
53      public static void main(String[] args) {
54          NotificationSender baseEmail = new EmailSender();
55
56          System.out.println("--- Порядок 1: Сначала Логирование,
    потом Retry ---");
57          // Читаем справа налево: Retry оборачивает Email,
    Логирование оборачивает Retry
58          NotificationSender order1 = new LoggingSender(new
    RetrySender(baseEmail));
59          order1.send("Привет");
60
61          System.out.println("\n--- Порядок 2: Сначала Retry, потом
    Логирование ---");
62          // Retry оборачивает (Логирование оборачивает Email)
63          NotificationSender order2 = new RetrySender(new
    LoggingSender(baseEmail));
64          order2.send("Привет");
65      }
66  }
```

Листинг – 23 8/Main.java

```
1   interface CourseReader {
2       String findById(int id);
3   }
4
5   interface CourseWriter extends CourseReader {
6       void save(String course);
7   }
8
9   class DatabaseRepository implements CourseWriter {
10      public void save(String course) {
11          System.out.println("Сохранено в БД: " + course);
12      }
13
14      public String findById(int id) {
15          return "Курс из БД";
16      }
17  }
18
19  class ReadOnlyRepository implements CourseReader {
20      public String findById(int id) {
21          return "Курс из кэша";
22      }
23  }
24
```

```
25  public class Main {
26      public static void main(String[] args) {
27          CourseReader reader = new ReadOnlyRepository();
28          System.out.println(reader.findById(1));
29
30          CourseWriter writer = new DatabaseRepository();
31          writer.save("Java");
32          System.out.println(writer.findById(1));
33      }
34  }
```

Листинг – 24 9/Main.java

```
1   class Student { int age; Student(int age) { this.age = age; } }
2   class Course { }
3
4   interface EnrollmentPolicy {
5       boolean isAllowed(Student s, Course c);
6   }
7
8   // Правило 1: По возрасту (18+)
9   class AgePolicy implements EnrollmentPolicy {
10      public boolean isAllowed(Student s, Course c) { return s.age >=
    18; }
11  }
12
13  // Правило 2: По пререквизитам (допустим, всегда true для простоты)
14  class PrereqPolicy implements EnrollmentPolicy {
15      public boolean isAllowed(Student s, Course c) { return true; }
16  }
17
18  // Правило 3: По вместимости (допустим, всегда true)
19  class CapacityPolicy implements EnrollmentPolicy {
20      public boolean isAllowed(Student s, Course c) { return true; }
21  }
22
23  // Клиент
24  class EnrollmentService {
25      private EnrollmentPolicy policy;
26
27      EnrollmentService(EnrollmentPolicy policy) { this.policy =
    policy; }
28
29      void enroll(Student s, Course c) {
30          if (policy.isAllowed(s, c)) System.out.println("Записан!");
31          else System.out.println("Отказ!");
```

```
32        }
33    }
34
35    public class Main {
36        public static void main(String[] args) {
37            Student kid = new Student(16);
38            Student adult = new Student(20);
39            Course c = new Course();
40
41            // Меняем стратегию на лету
42            EnrollmentService service = new EnrollmentService(new
      AgePolicy());
43            service.enroll(kid, c);   // Отказ
44            service.enroll(adult, c); // Записан
45        }
46    }
```

Листинг – 25 10/main_1.java, 10/main_2.java

```
1     interface EmailNotifier {
2         default void send() {
3             System.out.println("Отправка по Email");
4         }
5     }
6
7     interface SmsNotifier {
8         default void send() {
9             System.out.println("Отправка по SMS");
10        }
11    }
12
13    // КОМПИЛЯТОР ЗДЕСЬ ОСТАНОВИТСЯ С ОШИБКОЙ
14    class MultiNotifier implements EmailNotifier, SmsNotifier {
15    }
16
17    public class main_1 {
18        public static void main(String[] args) {
19            new MultiNotifier().send();
20        }
21    }


      // ===== 10/main_2.java =====
22    interface EmailNotifier {
23        default void send() {
24            System.out.println("Отправка по Email");
25        }
```

```
26    }
27
28    interface SmsNotifier {
29        default void send() {
30            System.out.println("Отправка по SMS");
31        }
32    }
33
34    // МЫ ЯВНО РАЗРЕШАЕМ КОНФЛИКТ
35    class MultiNotifier implements EmailNotifier, SmsNotifier {
36        @Override
37        public void send() {
38            // Явно указываем, чьи методы вызывать. Можно оба, можно
      один.
39            EmailNotifier.super.send();
40            SmsNotifier.super.send();
41        }
42    }
43
44    public class main_2 {
45        public static void main(String[] args) {
46            new MultiNotifier().send();
47        }
48    }
```

Листинг – 26 11/Main_1.java, 11/Main_2.java

```
1     class Point {
2         protected final int x, y;
3         public Point(int x, int y) { this.x = x; this.y = y; }
4
5         @Override
6         public boolean equals(Object o) {
7             if (this == o) return true;
8             if (!(o instanceof Point)) return false;
9             Point p = (Point) o;
10            return x == p.x && y == p.y;
11        }
12    }
13
14    class ColoredPoint extends Point {
15        private final String color;
16        public ColoredPoint(int x, int y, String color) {
17            super(x, y);
18            this.color = color;
19        }
```

```
20
21        @Override
22        public boolean equals(Object o) {
23            if (this == o) return true;
24            // Требуем, чтобы объект был именно ColoredPoint
25            if (!(o instanceof ColoredPoint)) return false;
26            ColoredPoint cp = (ColoredPoint) o;
27            return super.equals(o) && color.equals(cp.color);
28        }
29    }
30
31    public class Main_1 {
32        public static void main(String[] args) {
33            Point p = new Point(1, 2);
34            ColoredPoint cp = new ColoredPoint(1, 2, "Red");
35
36            System.out.println("p.equals(cp): " + p.equals(cp));  //
      true (Point не видит цвет)
37            System.out.println("cp.equals(p): " + cp.equals(p));  //
      false (ColoredPoint требует цвет)
38            System.out.println("Симметрия нарушена!");
39        }
40    }


      // ===== 11/Main_2.java =====
41    class Point {
42        private final int x, y;
43        public Point(int x, int y) { this.x = x; this.y = y; }
44
45        @Override
46        public boolean equals(Object o) {
47            if (this == o) return true;
48            if (o == null || getClass() != o.getClass()) return false;
      // Строгое сравнение
49            Point point = (Point) o;
50            return x == point.x && y == point.y;
51        }
52    }
53
54    // ColoredPoint БОЛЬШЕ НЕ НАСЛЕДУЕТ Point. Он содержит его внутри.
55    class ColoredPoint {
56        private final Point point;
57        private final String color;
58
59        public ColoredPoint(int x, int y, String color) {
60            this.point = new Point(x, y);
```

```
61            this.color = color;
62        }
63
64        @Override
65        public boolean equals(Object o) {
66            if (this == o) return true;
67            if (o == null || getClass() != o.getClass()) return false;
68            ColoredPoint that = (ColoredPoint) o;
69            return point.equals(that.point) && color.equals(that.color);
70        }
71    }
72
73    public class Main_2 {
74        public static void main(String[] args) {
75            Point p = new Point(1, 2);
76            ColoredPoint cp = new ColoredPoint(1, 2, "Red");
77
78            System.out.println("p.equals(cp): " + p.equals(cp));  //
      false (разные классы)
79            System.out.println("cp.equals(p): " + cp.equals(p));  //
      false (разные классы)
80            System.out.println("Симметрия соблюдена!");
81        }
82    }
```

Листинг – 27 12/Main.java

```
1     // 1. Запечатанный интерфейс (разрешены ТОЛЬКО эти три класса)
2     sealed interface EnrollmentResult permits Accepted, Rejected,
      WaitListed {}
3
4     // 2. Разрешенные реализации
5     record Accepted(String courseName) implements EnrollmentResult {}
6     record Rejected(String reason) implements EnrollmentResult {}
7     record WaitListed(int position) implements EnrollmentResult {}
8
9     public class Main {
10        public static void main(String[] args) {
11            EnrollmentResult result1 = new Accepted("Java");
12            EnrollmentResult result2 = new Rejected("Нет мест");
13            EnrollmentResult result3 = new WaitListed(5);
14
15            checkResult(result1);
16            checkResult(result2);
17            checkResult(result3);
18        }
19
```

```
20        // 3. Exhaustive switch (исчерпывающий)
21        public static void checkResult(EnrollmentResult result) {
22            switch (result) {
23                case Accepted a -> System.out.println("Успех! Вы
      записаны на " + a.courseName());
24                case Rejected r -> System.out.println("Отказ. Причина: "
      + r.reason());
25                case WaitListed w -> System.out.println("В листе
      ожидания. Ваш номер: " + w.position());
26
27                // ОБРАТИ ВНИМАНИЕ: тут НЕТ default!
28                // Компилятор сам знает, что других вариантов быть не
      может.
29            }
30        }
31    }
```

Листинг – 28 13/Main.java

```
1     // 1. Интерфейс (единый контракт)
2     interface NotificationSender {
3         void send(String message);
4     }
5
6     // 2. Реализации
7     class EmailSender implements NotificationSender {
8         @Override
9         public void send(String message) {
10            System.out.println("Email: " + message);
11        }
12    }
13
14    class SmsSender implements NotificationSender {
15        @Override
16        public void send(String message) {
17            System.out.println("SMS: " + message);
18        }
19    }
20
21    class PushSender implements NotificationSender {
22        @Override
23        public void send(String message) {
24            System.out.println("Push: " + message);
25        }
26    }
27
28    // 3. Клиент (никаких switch)
```

```
29  class NotificationService {
30      public void send(NotificationSender sender, String message) {
31          sender.send(message);
32      }
33  }
34
35  // 4. Запуск
36  public class Main {
37      public static void main(String[] args) {
38          NotificationService service = new NotificationService();
39
40          service.send(new EmailSender(), "Привет");
41          service.send(new SmsSender(), "Привет");
42          service.send(new PushSender(), "Привет");
43      }
44  }
```

Листинг – 29 14/Main.java

```
1   // Ось 1: Формат
2   interface Format { String name(); }
3   class Online implements Format { public String name() { return
    "Online"; } }
4   class Classroom implements Format { public String name() { return
    "Classroom"; } }
5   class Hybrid implements Format { public String name() { return
    "Hybrid"; } }
6
7   // Ось 2: Оплата
8   interface Payment { String name(); }
9   class Free implements Payment { public String name() { return
    "Free"; } }
10  class Fixed implements Payment { public String name() { return
    "Fixed"; } }
11  class Subscription implements Payment { public String name() {
    return "Subscription"; } }
12
13  // Сам курс (собирает две оси)
14  class Course {
15      String title; Format format; Payment payment;
16
17      Course(String title, Format format, Payment payment) {
18          this.title = title; this.format = format; this.payment =
    payment;
19      }
20
21      void printInfo() {
```

```
22          System.out.println(title + " | " + format.name() + " | " +
    payment.name());
23      }
24  }
25
26  public class Main {
27      public static void main(String[] args) {
28          // 1. Существующие комбинации
29          new Course("Java", new Online(), new Free()).printInfo();
30          new Course("Python", new Classroom(), new
    Subscription()).printInfo();
31
32          // 2. Добавляем новый формат (VR) — всего 1 класс
33          Format vr = new Format() { public String name() { return
    "VR"; } };
34          new Course("C#", vr, new Free()).printInfo();
35
36          // 3. Добавляем новую оплату (Crypto) — всего 1 класс
37          Payment crypto = new Payment() { public String name() {
    return "Crypto"; } };
38          new Course("Rust", new Online(), crypto).printInfo();
39      }
40  }
```

Листинг – 30 15/Main.java

```
1   sealed interface RuleResult permits Ok, Fail {}
2   record Ok() implements RuleResult {}
3   record Fail(String reason) implements RuleResult {}
4
5   interface Rule {
6       RuleResult check(String student);
7   }
8
9   class AgeRule implements Rule {
10      public RuleResult check(String student) {
11          return student.equals("Вася") ? new Fail("Слишком молод") :
    new Ok();
12      }
13  }
14
15  class PaymentRule implements Rule {
16      public RuleResult check(String student) {
17          return student.equals("Иван") ? new Fail("Нет денег") : new
    Ok();
18      }
19  }
```

```
20
21  class AllOf implements Rule {
22      private final Rule[] rules;
23      AllOf(Rule... rules) { this.rules = rules; }
24
25      public RuleResult check(String student) {
26          for (Rule r : rules) {
27              RuleResult res = r.check(student);
28              if (res instanceof Fail f) return f;
29          }
30          return new Ok();
31      }
32  }
33
34  class AnyOf implements Rule {
35      private final Rule[] rules;
36      AnyOf(Rule... rules) { this.rules = rules; }
37
38      public RuleResult check(String student) {
39          for (Rule r : rules) {
40              if (r.check(student) instanceof Ok) return new Ok();
41          }
42          return new Fail("Ни одно правило не пройдено");
43      }
44  }
45
46  class EnrollmentService {
47      private final Rule rule;
48      EnrollmentService(Rule rule) { this.rule = rule; }
49
50      void enroll(String student) {
51          RuleResult result = rule.check(student);
52          switch (result) {
53              case Ok o -> System.out.println("УСПЕХ: " + student + "
    записан!");
54              case Fail f -> System.out.println("ОТКАЗ: " + student +
    " (" + f.reason() + ")");
55          }
56      }
57  }
58
59  public class Main {
60      public static void main(String[] args) {
61          Rule age = new AgeRule();
62          Rule payment = new PaymentRule();
```

```
63
64            Rule strictRule = new AllOf(age, payment);
65            Rule lenientRule = new AnyOf(age, payment);
66
67            System.out.println("--- AllOf ---");
68            new EnrollmentService(strictRule).enroll("Вася");
69            new EnrollmentService(strictRule).enroll("Иван");
70            new EnrollmentService(strictRule).enroll("Петя");
71
72            System.out.println("\n--- AnyOf ---");
73            new EnrollmentService(lenientRule).enroll("Вася");
74            new EnrollmentService(lenientRule).enroll("Иван");
75        }
76    }
```